Understanding ML, Chapter 1: What Does It Mean to Learn?

After reading: you can say what learning is, when to write rules and when to learn, which paradigm a task belongs to and why, what the four ingredients of any learning system are, why finite data never determine the rule, how a shortcut is born and how to catch it, and why cross-entropy is the loss of classification

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

A machine can be given an ability in two ways: a human writes the procedure, or the machine is given *experience* and a *capacity to learn* and finds the procedure itself. The second route exists because people can do many things (read a handwritten 3, spot a wolf) that they cannot describe. Experience reaches the machine as examples, observations or interactions, and the kind of feedback that reaches the learner decides whether the problem is supervised, unsupervised, self-supervised or reinforcement learning. Any learning system can be described by four ingredients: the experience, the model, the learning algorithm and the evaluation criterion. The thing we actually want is *generalization*, good behaviour on situations never seen, and the chapter proves why the training data can never guarantee it (finitely many examples agree with infinitely many rules) and shows the typical failure, the *shortcut*: a cue that predicts the labels in the collection but not in the world. The chapter closes with the one inequality (Gibbs) that explains why cross-entropy will be the training criterion for classification. What is manipulated throughout: the experience (which examples, how diverse, which labels), the representation (which numbers the model sees), and the rules by which success is judged.

Contents

1 Learning from experience **2**

2 Two routes to machine ability **3**

3 What counts as experience? The three paradigms **5**

4 Supervised learning: examples, labels, outputs **8**

5 Representations: what is presented to the learner **11**

6 Four ingredients of a learning system **14**

7 The challenge of generalization **16**

8 A connection to information theory **20**

Coverage of the textbook chapter **23**

1 Learning from experience

Learning, capacity to learn, experience

Learning is the process through which experience changes a learner and develops what it can do. Two things are needed: **experience** (observation, practice, interaction, instruction, feedback) and a **capacity to learn** (a stone exposed to a thousand piano lessons learns nothing). For animals the capacity comes from the brain; for a machine it comes from design choices. The chapter's picture is

$$\text{Capacity to Learn} + \text{Experience} \xrightarrow{\text{Learning}} \text{Acquired Abilities.}$$ [not on your sheet: know it]

The abilities we care about are four overlapping activities, a *vocabulary*, not a taxonomy: **recognise** (what is it?), **predict** (what may happen?), **reason** (what follows?), **act** (what should I do?). Language draws on all four at once.

Purpose. The definition separates the two things a designer controls: what the system experiences, and what in the system can change in response.

Impact. Change the experience and the same learner acquires a different ability; remove the capacity to change and no amount of experience helps. Everything later in the course is a way of engineering one of the two.

One task, four abilities: a radiologist

Reading a chest X-ray, a radiologist *recognises* anatomical structures and abnormal shadows, *predicts* how a lesion may evolve, *reasons* from symptoms and image to a differential diagnosis, and *acts* by ordering a test or a treatment. A machine-learning system may take on any subset: a classifier flagging suspicious regions is recognition; a model of tumour growth is prediction; a treatment-recommendation system is decision and action. Keeping the four apart lets you state precisely what the system is supposed to do, which is the first question of any project.

Using the four abilities as a classification scheme

Students write "this task is recognition, therefore it is supervised learning" or treat the four words as exhaustive categories of intelligence. The notes say explicitly that they are *not* an exhaustive classification. They are words for describing what an ability does; the learning paradigm (next section) is decided by the feedback, not by which of the four verbs fits.

Key insight. Learning is a change in the learner caused by experience; a machine learns only if something in it can change and something in its experience tells it which change is good.

How the professor asks it

Q. *Define learning in the sense of the course and explain why "exposure to data" is not by itself learning. Give one example of a system that has experience but no capacity to learn.*
A. Learning is the process by which experience changes a learner and develops what it can do. Exposure alone is not learning because the second ingredient, a capacity to change in response to the experience, may be absent: the stone that attends a thousand piano lessons is exposed to the same experience as the student and acquires nothing. A fixed rule-based spam filter that receives millions of emails but whose rules never change has experience and no capacity to learn; its error rate on new spam does not move. The consequence for design is that a learning system

must specify *what can change* (the model's parameters, possibly its representation) and *what information drives the change* (the evaluation signal).

Q. *A hospital wants "an AI for radiology". Using the four abilities, show why the request is underspecified and rewrite it as two distinct, well-posed tasks.*
A. The request names no ability. Reading an X-ray involves recognising structures, predicting evolution, reasoning to a diagnosis and acting on it, and a system can take on any subset; each subset needs different experience and a different evaluation. Task 1 (recognition): flag regions of a chest X-ray that look abnormal, trained on images with marked regions, judged by how many abnormalities are flagged and how many false flags are raised. Task 2 (prediction): estimate the probability that a lesion grows within six months, trained on pairs of scans six months apart, judged on later patients. The two tasks share the images and nothing else; conflating them makes the evaluation criterion impossible to state.

2 Two routes to machine ability

Route 1: encode human knowledge

Humans learn first, then transfer the result to the machine as explicit instructions:

$$\text{Human Knowledge} \longrightarrow \text{Explicit Rules or Procedures} \longrightarrow \text{Computer.}$$ [not on your sheet: know it]

Pascal's seventeenth-century Pascaline embodied arithmetic in a mechanism; a program embodies a tax law, merge sort, or "hash the password and compare". Early AI pushed the route into expert domains: an **expert system** such as MYCIN stored knowledge elicited from specialists as explicit facts and `IF ...THEN ...` rules and could reason well in a restricted domain. As systems grew, extracting, expressing and maintaining the knowledge became the hard part: the **knowledge acquisition bottleneck**.

Purpose. Exact, auditable behaviour when a reliable procedure is known.

Impact. No data are needed and no approximation is introduced; but the system knows only what was written, and cannot improve from what it observes.

Route 2: make the machine learn

When useful experience is available and a complete procedure is hard to specify, give the machine the experience and a capacity to learn, and let it acquire the ability itself. The question changes from "what rules should humans write?" to "how can we give a machine the capacity to learn from experience?". Designing computational methods that make this possible is **machine learning**. The capacity to learn is provided by design choices: what the system can represent, what can change through experience, what information guides the changes, and what counts as good performance. Machine learning *still involves programming*: humans build the learner, the data pipeline and the decision logic; only the task-specific behaviour is acquired from data.

Purpose. Abilities we can recognise but cannot describe.

Impact. Behaviour becomes approximately correct and data-dependent: it improves with better experience and degrades when the world drifts away from the experience.

The rules-versus-examples experiment on handwritten 3s

Setup. A 20×20 binary image; decide whether it shows a 3. Route 1 writes rules: (R1) two bumps opening left, i.e. the rightmost black pixel of the top half and of the bottom half both lie in the right third; (R2) the middle row has black pixels only in the right half (the waist); (R3) between 40 and 90 black pixels; (R4) no black pixel touches the left border. Route 2 collects a

few thousand labelled 3s and non-3s and lets a learner find regularities.

Held fixed. The task (is it a 3?), the image size, the set of test images.

What we manipulate. The way the ability is produced (rules versus examples), and the *perturbations* applied to the digit: shift 4 pixels right, write it narrowly, rotate by 15°, use a thick stroke.

What we measure. For each perturbation, which rules still hold; for the learner, the error on perturbed test images, and the cost of handling a new kind of variation (editing rules versus adding examples).

What the plot shows. A table of perturbations against rules: shift breaks R2 (the waist crosses the centre column), narrow writing breaks R1 (bumps no longer reach the right third), rotation breaks R2 (the waist leaves the middle row), a thick stroke breaks R3 (the pixel count doubles). Every fix breeds a new failure: "right third" becomes "right third of the bounding box", corrupted by a stray dot; "middle row" becomes "row of minimum width between the bumps", which also matches 5s and 8s; the pixel range widens until it excludes nothing.

How to read it. Rules pile up with thresholds nobody knows how to set and interactions nobody can reason about, while the learner absorbs each variation from examples that already contain it. The experiment reveals the essential difference: explicit programming requires us to *know and state* the procedure; learning requires only that we can *recognise correct behaviour on examples*.

Programs that should stay programs, and one that should not

Income tax: the rule is written in law; a program applies it exactly; a learned model would approximate the law from past returns and be wrong on rare cases, and nobody wants an approximately correct tax bill. *Sorting*: merge sort is proved correct for every input. *Password check*: "hash and compare" must be right every time, which learning never guarantees. By contrast, an industrial fault-diagnosis expert system built from two hours a week with an engineer runs into three separate difficulties: *obtaining* the knowledge (the engineer "just knows" that a rattle means a loose coupling and cannot state it as a test), *representing* it (judgements are graded, interacting, full of exceptions; hundreds of rules contradict each other in unanticipated corners), and *unfamiliar situations* (a new generation of machines shows faults the engineer never saw; the rules are silent and cannot improve). Learning attacks all three: patterns come from logged sensor readings and repair records, graded effects are numbers, and the model is retrained on the new machines' data. Expert knowledge still decides which measurements to record, sanity-checks the model, and covers rare catastrophic faults with no data.

Writing rules versus learning from experience

	Route 1: rules	Route 2: learning
What is transferred	the *result* of human learning, as a procedure	the *experience*, plus a capacity to learn
Requirement	someone can state the complete procedure	someone can recognise correct behaviour on examples, and examples exist
What is manipulated	the rule base	the experience, the model family, the objective
Behaviour	exact, only on anticipated cases	approximate, adapts to the cases in the data
Failure mode	knowledge acquisition bottleneck; silent on unfamiliar cases	wrong or narrow experience teaches the wrong lesson confidently
Prefer when	procedure known, exactness required (tax, sorting, crypto, safety interlocks)	ability without describable procedure, many examples (digits, speech, fraud)

Most real systems mix both: a learned model produces a prediction and an explicit program decides what to do with it under fixed business rules.

"Machine learning replaces programming" and "expert knowledge is obsolete"

Both are false and both cost points. Learning replaces the *task-specific rules*; the learner, the pipeline and the decision logic are still programmed. Expert knowledge decides what to measure, checks whether the model's behaviour is physically plausible, and covers cases for which no data exist. The tempting error is to answer "learn it" to every task; the three questions to ask are: can a competent person write the complete procedure (then program it); can we recognise correct behaviour on examples and collect many (then learn it); what does a mistake cost (learning is approximately correct; when that is unacceptable, keep the program or wrap the learned part in explicit safeguards).

Key insight. Having an ability is not the same as having an explicit procedure that describes it; the whole field rests on this gap, and human vision is its largest example.

How the professor asks it

Q. *At what point in the handwritten-digit thought experiment would you stop adding rules and start giving the computer examples? What does the experiment reveal about the difference between the two routes? (Exercises ch. 1, Q1(d))*
A. When the cost of writing and maintaining rules exceeds the cost of collecting labelled examples, which happens as soon as each new batch of handwriting exposes new exceptions and each rule change has unpredictable side effects on the others. Mechanism: rules must anticipate every variation (shift, scale, slant, stroke width) explicitly, with thresholds nobody can set; examples already contain the variations, and the learner absorbs them. The experiment reveals that the two routes have different *requirements*: programming needs the procedure to be known and stated, learning needs only that correct behaviour be recognisable on examples. Consequence: route 1 fails exactly on the abilities humans find easiest, because those are the ones we perform without a describable procedure.

Q. *Give a counterexample to the claim "if a task is important and we have a lot of data, we should learn it from the data".*
A. Income tax, or a password check. The procedure is completely known (written in law; "hash and compare"), so a program applies it exactly; a learned model would only approximate it from past cases and be wrong on rare ones, and "approximately correct" is unacceptable there. Mechanism: learning gives behaviour that is correct on average over situations resembling the data, never a guarantee on every input. Consequence: the decision is made by three questions (is the procedure known; can correct behaviour be recognised on examples; what does a mistake cost), not by the importance of the task or the volume of data.

3 What counts as experience? The three paradigms

Examples, observations, interactions

Experience reaches a machine in three forms. **Examples** pair an input with a desired output (an image with its digit, a house with its sale price): *direct guidance* about what the output should be. **Observations** are data without a supplied answer (images, text, audio, sensor logs): they contain *regularities* that can be exploited without anyone labelling them. **Interactions** arise when an agent acts in an environment and observes what follows: the learner *partly shapes its own future experience*, because its actions change what happens next.

Purpose. To know, before any algorithm is chosen, what information the learner will receive; that decides what kind of learning is even possible.

Impact. The same raw data (X-rays, network flows) can be presented as any of the three forms, and the form changes which questions can be answered and how success is judged.

The three learning settings

Supervised learning: experience consists of examples with desired outputs. **Unsupervised learning**: the learner searches for useful structure in observations without desired outputs; **self-supervised learning** also learns from observations but *constructs* a learning signal from them (hide a part, predict it). **Reinforcement learning**: experience comes through interaction; consequences and rewards provide information about the quality of behaviour. These are broad settings, not mutually exclusive kinds of system; one system can use several.

Purpose. A paradigm names *what information about "doing well" reaches the learner*: a correct answer per example, nothing external, an answer manufactured from the data itself, or a reward earned through action.

Impact. Supervision is the most informative signal and the most expensive to obtain; observations are cheap and plentiful but carry no answer; rewards are delayed and evaluative ("that was good", not "this was the right answer") and the learner's own actions bias what it sees next.

Same data, four paradigms: the hospital experiment

Setup. One collection of chest X-rays; for some patients a diagnosis was recorded. Run four projects on it.

Held fixed. The images themselves.

What we manipulate. What is presented to the learner and what feedback it gets: (1) image plus recorded diagnosis, feedback is agreement with the diagnosis; (2) images only, feedback is an internal criterion such as within-group similarity; (3) images only, but a region is hidden and the model must reconstruct it, feedback is the match with the hidden pixels; (4) a robotic device positions the sensor and receives a reward for good positioning and a penalty for unsafe movements.

What we measure. For (1) accuracy against diagnoses on new patients; for (2) the quality of the grouping by the internal criterion and by whether a clinician finds the groups meaningful; for (3) reconstruction error on hidden regions of new images; for (4) the reward collected on new positioning episodes.

What the plot shows. There is no single curve; the point is a table with one row per project and columns "experience", "feedback", "paradigm": supervised, unsupervised, self-supervised, reinforcement.

How to read it. The paradigms differ not by the data type (all four use X-rays) but by what information about doing well reaches the learner. Only in (4) does the learner influence its own future experience. "Unsupervised" does not mean "no objective": project (2) still needs a criterion for what a good grouping is; what is absent is a human-supplied answer per example.

Same records, three questions; one cybersecurity task per paradigm

A university has ten years of student records. *Will each student graduate?* Supervised, categorical output: classification. *What will the final grade be?* Supervised, numerical output: regression. *Which students have similar trajectories?* No desired output: unsupervised. The collection supports several problems because the *problem* is defined by the question asked, not by the data. In security: supervised, classify flows as benign or attack with the label of each training flow as the signal; self-supervised, predict a masked field of a flow from the rest, the hidden part

is the signal and no human labelled anything; reinforcement, an agent chooses block, rate-limit or nothing in a simulated network, the signal is a reward (attack contained minus disruption caused) and its actions change the traffic it sees next.

Supervised versus unsupervised versus self-supervised versus reinforcement

	Supervised	Unsupervised	Self-supervised	Reinforcement
Experience	examples (x, y)	observations x	observations x	actions and consequences
Signal	the desired output per example	an internal criterion (similarity, density)	a target built from the data itself (the hidden part)	a reward, delayed and evaluative
Who shapes the experience	the data collector	the data collector	the designer of the pretext task	partly the learner's own actions
Typical output	a prediction for a new x	a structure (groups, factors)	a representation, or a generator	a policy from states to actions
Failure mode	bad or narrow labels	meaningless groups under the wrong criterion	pretext task that does not transfer	reward hacking; exploring unsafe actions

Naming the paradigm from the data type

"It uses images, so it is supervised" or "it uses logs, so it is unsupervised". The paradigm is decided by the feedback that reaches the learner: four projects on the same X-rays belong to four paradigms. A related slip: calling self-supervision "supervised because there is a target". There is a target, but it was manufactured from the observation itself; nobody labelled anything, which is exactly why it scales to the whole internet.

Key insight. Ask what information about "doing well" reaches the learner; that single question names the paradigm.

How the professor asks it

Q. *A hospital holds X-rays, some with recorded diagnoses. Classify the following four projects into learning settings and justify each: predict pneumonia on a new X-ray; group thousands of unlabelled X-rays by visual similarity; hide a region of each X-ray and predict it; teach a robot to position the sensor with rewards. (Exercises ch. 1, Q5)*

A. (1) Supervised: experience is images paired with diagnoses and the feedback is agreement with the recorded diagnosis. (2) Unsupervised: images only; the only feedback is an internal criterion such as within-group similarity, no external answer per image. (3) Self-supervised: images only, but each image supplies its own target (the hidden region), so a supervised-looking problem is built without labels. (4) Reinforcement: experience is the device's own actions and their consequences; the feedback is a delayed, evaluative reward, not a correct answer, and only here does the learner shape what it sees next. The data are identical in all four; the paradigm is decided by what information about doing well reaches the learner.

Q. *Explain why "unsupervised" does not mean "no objective", with an example.*

A. Unsupervised means no human supplies the desired answer for each example; it does not mean the learner has nothing to optimise. Grouping X-rays still requires a criterion for a good grouping (for instance, images within a group should be more similar to each other than to images in other groups), and the learner judges structure by that internal criterion. Change the criterion and the groups change, so the designer still chooses an objective. Consequence: an unsupervised result

is only as meaningful as its criterion, and "the algorithm found these clusters" is not evidence that the clusters mean anything clinically.

Q. *Why is reinforcement learning the only setting in which the learner influences its own future experience, and what difficulty does that create?*
A. In supervised and unsupervised learning the data are collected independently of the learner; in reinforcement learning the agent's actions change what happens next, so the experience it accumulates depends on the policy it currently has. The difficulty is twofold: the reward is delayed and evaluative, so the learner is never told which action was right, only roughly how good a sequence was; and a poor early policy visits poor situations, so the learner can fail to discover better behaviour (and may explore unsafe actions, which is why the sensor-positioning robot is penalised for unsafe movements). Consequence: data collection and learning cannot be separated as they can in the other settings.

4 Supervised learning: examples, labels, outputs

Supervised example, training dataset

A **supervised example** is a pair (x, y) of an **input** x and a **desired output** y; a **training dataset** is a finite collection $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ [on your sheet]. The desired output is called the **target**, the **label** or the **ground truth**. The learner's goal is a model that produces useful outputs for *new* inputs, not for the N it has seen.

Purpose. The pair is the unit of direct guidance: it says, for one input, what the right answer was.

Impact. Every property of the pairs (how many, how varied, how the y were obtained, how old they are) limits what can be learned; no algorithm compensates for bad pairs.

Where labels come from, and why "ground truth" is a strong phrase

Labels come from three imperfect sources. **Measurement**: temperature, electricity consumption, a recorded sale price; instrument error and recording mistakes. **Annotation**: identifying an object in an image, interpreting a scan; experts disagree, and labels depend on conventions fixed when the task was defined (is "dog" correct for a wolf-dog hybrid?). **Delayed outcome**: whether a loan is repaid, whether a machine fails within a month; known only after time has passed, hence only for old inputs, in a world that may have changed since.

Purpose. To treat label quality as an explicit limit on achievable accuracy.

Impact. A model trained on labels that are wrong 10% of the time cannot be expected to agree with the (noisy) labels more than about 90% of the time, and its errors will partly mirror the annotators' errors. A reported 99% accuracy on labels that are themselves 90% reliable should raise suspicion, not admiration.

The training cycle, as a procedure

Setup. A model with adjustable parameters, a training dataset of pairs.

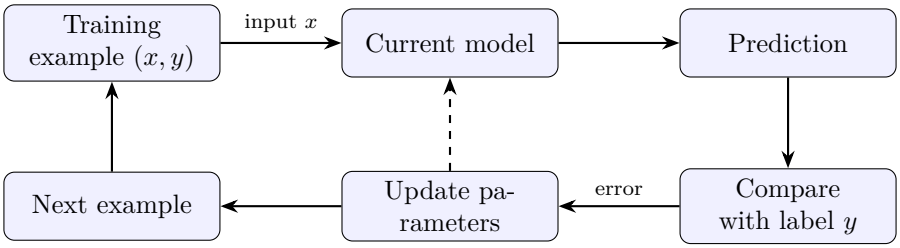
Held fixed. The dataset, the form of the model, the rule that compares a prediction with a label.

What we manipulate. The model's parameters, one update at a time.

What we measure. The disagreement between the prediction and the label on the current example; aggregated over the data, the training error.

What the plot shows. Training error against the number of updates: typically falls, fast at first and then slowly.

How to read it. The cycle is: receive an input, produce a prediction, compare with the desired output, use the comparison to change the model, move to the next example, repeat across the available experience. This computational process is **training**. Chapter 2 turns the informal cycle into a mathematical problem (a loss, an empirical risk, a minimisation); the falling curve says only that the model fits what it has seen, which, as Section 7 shows, is not the goal.



Phishing detection: designing the experience before any algorithm

Where do examples come from? Reported phishing pages (security feeds, user reports) and legitimate pages (popular sites, the organisation's own traffic, and the real login pages that phishing imitates, which are the hard negatives). *How varied?* Many languages, many impersonated brands, pages built with different kits, crude and sophisticated attacks; otherwise the model learns "phishing = badly spelled English page imitating one bank". *How are correct outcomes established?* Analyst confirmation or confirmation by the impersonated organisation; both imperfect, and over-eager reporters mislabel some legitimate pages. *How fast does the world change?* Monthly; labels from three years ago describe attacks nobody uses, and today's attacks are absent. A million pages from one feed, one language and one year is a very large dataset that teaches a narrow, outdated notion of phishing with great confidence. Volume does not substitute for coverage.

Regression and classification

A supervised problem is **regression** when the desired output is numerical (a real number or a vector) and **classification** when it is a category from a finite set. *The output decides the type, not the input.* Richer outputs exist (object detection returns boxes with labels, segmentation a class per pixel, translation a sentence); they are still supervised, with structured outputs.

Purpose. The output type decides which losses make sense, which model outputs are needed, and how to evaluate.

Impact. The same medical image gives regression (tumour size in millimetres) or classification (benign or malignant); the same weather data gives regression (millimetres of rain) or classification (will it rain?).

Prediction, decision, action

A **prediction** expresses what the model says about an unknown output. A **decision** uses the available information together with a *criterion* to select a response. An **action** is what is carried out when the decision calls for something to be done.

Purpose. To separate what the data say from what should be done about it; the second depends on costs the data do not contain.

Impact. A model predicts a 12% probability of disease; a protocol recommends testing above 10% (decision); the test is scheduled (action). Change the threshold to 20% and the same prediction leads to a different decision. The three coincide only when all errors cost the same; they separate as soon as a missed disease costs more than an unnecessary test (Chapter 2 makes this exact).

Regression versus classification; prediction versus decision

	Regression	Classification
Output	a number or vector (price in MAD, millimetres, hours, megawatts)	one of a finite set of classes (digit, spam or not, species)
What "close" means	numerical distance between y and the prediction	right class or wrong class; possibly a probability over classes
Decided by	the output, never the input	the output, never the input
Trap	"regression" does not mean linear regression	"classification" does not mean two classes
	Prediction	**Decision**
Object	what the model says (a number, a class, a probability)	a response chosen by applying a criterion (threshold, costs, protocol)
Changes when	the data or the model change	the costs or the protocol change, with the same prediction

A label is not truth; a probability is not a decision

Three slips. (1) Trusting an accuracy figure without asking how the labels were produced: measured, annotated or observed later, each with its own error, and label error caps what any model can reach. (2) Reading "regression" as "linear" or "classification" as "binary". (3) Writing "the model says yes" when the model says 0.6: the model has made a prediction, and the decision needs a threshold that encodes costs; two users with different costs should decide differently on the same 0.6.

Key insight. Supervised learning is only as good as its pairs: where the labels came from, how varied the inputs are, and how old the pairs are limit what can be learned before any algorithm is chosen.

How the professor asks it

Q. *Network flows are labelled benign or attack from incident metadata; the system scores a new flow and decides whether to alert. Identify the physical object, the representation x, the target y, the model output and the final action, and explain why the output and the action are different objects. (Tutorial 1, Q1(a) and (c))*

A. Physical object: the completed flow, the actual packet exchange between two endpoints. Representation x: the feature vector computed when the flow completes (duration, packets and bytes per direction, protocol, destination port, flags). Target $y \in \{0 = \text{benign}, 1 = \text{attack}\}$ from the ground-truthing process, so it describes the episode the flow belonged to, not its appearance. Model output: a score or a posterior estimate $\hat{p}(y = 1 \mid x)$. Action: alert or not, possibly a third action, send to an analyst. The output is a *prediction*, a number summarising what the features say; the alert is a *decision*, a rule applied to that number which also needs the costs of a false alarm and a miss. The same 0.3 should alert where a miss is far costlier than a false alarm and not where alert fatigue dominates; keeping them apart lets you change the threshold without retraining.

Q. *Some attack flows and some benign flows have exactly the same observed features. Explain why no classifier using these features can separate them, and whether the labels are therefore wrong. (Tutorial 1, Q1(b))*

A. It cannot, and the labels can still be right. A classifier is a function of x alone, so it gives both flows the same answer and is wrong on one of them whatever it does; the best possible rule predicts the majority class at that x and still loses the minority share. The labels are correct because "attack" is defined by episode membership: an ordinary-looking reconnaissance request is correctly labelled attack. What is missing is information in x (the host's other flows, the time

context), not correctness in y. Consequence: the fix is a richer representation, not relabelling, and no model beats this floor on the given features.

5 Representations: what is presented to the learner

Representation, feature engineering, representation learning

A learner never sees a house or a photograph; it sees numbers. A **representation** is a transformation of the original input into a form that makes useful structure easier for a learning model to exploit. The form of experience says what information *exists*; the representation says what information the model *gets*. Two pipelines:

$$\text{Raw Input} \longrightarrow \text{Human-Designed Features} \longrightarrow \text{Learned Predictor},$$
$$\text{Raw Input} \longrightarrow \text{Learned Representation} \longrightarrow \text{Learned Predictor}.$$[not on your sheet: know it]

Feature engineering (first pipeline): humans decide what to present (floor area, location, age for a house; edges, corners, textures for vision; mel-frequency cepstral coefficients for speech) and the model learns how to use it. **Representation learning** (second pipeline): the machine discovers the transformation from experience; the idea is broader and older than deep learning, which suits it because successive layers build higher-level features (parts, shapes) from lower-level ones (edges, local patterns). Humans still choose the architecture, the method, the experience and the objective.

Purpose. To control which information actually reaches the model; the choice can decide whether learning succeeds at all.

Impact. With few examples, a good engineered representation makes a problem learnable that is hopeless on raw input; with enormous data, a learned representation can surpass the engineered one because the designer forgot features that matter.

Two representations of the same email

Setup. A spam filter. Representation (a): the raw sequence of characters. Representation (b): a vector of counts (how often each of 10,000 common words appears, whether the sender is in the address book, the number of links, the number of exclamation marks).

Held fixed. The emails, the labels, the learning algorithm.

What we manipulate. The representation, and the number of training examples (a few thousand, then billions).

What we measure. Error on new emails.

What the plot shows. Error against training-set size, one curve per representation: with a few thousand examples (b) is learnable and (a) is hopeless; the (a) curve only comes down with billions of examples and a model able to build its own representation, and can then cross below (b).

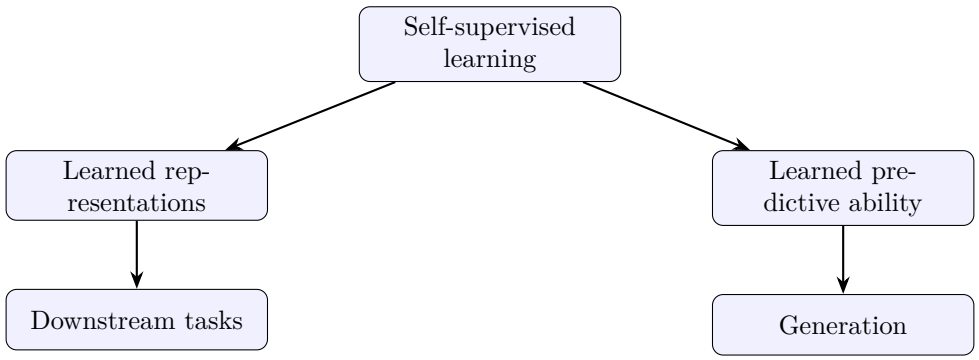
How to read it. Representation (b) makes "many links, no known sender, the word *prize*" directly visible as a few coordinates; (a) hides the same facts behind thousands of characters whose positions vary from message to message. The crossing shows the two regimes: engineered features win when data are scarce; learned representations win when data are abundant, because the designer of (b) forgot features that matter.

Self-supervised learning and generation

In **self-supervised learning** a learning problem is constructed from the observations themselves: part of an observation is used to predict another part. Hide a word in "The train to _____ leaves at nine"; mask a square of a photograph and reconstruct it; remove half a second of speech and predict it. Nobody labelled anything; the data are their own supervision. Two uses of what is learned: the *representation* can be adapted with a small labelled dataset to classification or detection (*downstream tasks*); or the *predictive ability* is useful directly. An **autoregressive language model** learns to predict the next token from the previous ones; once it can predict, it can **generate**: predict a next token, append it, predict again.

Purpose. To learn from unlabelled observations, which exist in vastly larger quantities than labelled examples.

Impact. The pretext task gives the model a reason to discover regularities; the quality of the representation depends on whether the pretext task requires the structure that the downstream task needs.



The two branches are two *uses* of what was learned, not two kinds of model: a generative language model also learns internal representations.

A next-token model computed by hand, and what it generates

Corpus: `the cat sat on the mat . the cat ate the rat .` The simplest next-token model looks only at the previous word and estimates probabilities by counting. After `the` the next word is `cat` twice, `mat` once, `rat` once, four occurrences in all, so

$$\mathbb{P}(\texttt{cat} \mid \texttt{the}) = \tfrac{2}{4}, \qquad \mathbb{P}(\texttt{mat} \mid \texttt{the}) = \tfrac{1}{4}, \qquad \mathbb{P}(\texttt{rat} \mid \texttt{the}) = \tfrac{1}{4}.$$ [not on your sheet: know it]

After `cat`: `sat` and `ate`, each $\frac{1}{2}$. After `sat`: `on` with probability 1; after `on`: `the`; after `mat` or `rat`: the full stop. Each number is a count divided by the number of times the context word appeared; no label was written, the corpus itself says what followed what.

Generation. Start from `the`; sample from $(\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, say `cat`; sample from $(\frac{1}{2}, \frac{1}{2})$, say `ate`; after `ate` the only continuation is `the`; and so on. One possible output is `the cat ate the mat .`, a sentence that never appears in the corpus: the model recombined regularities it found.

Probability of a whole sentence. Multiply the transitions along the way. For `the cat sat on the mat .` started at `the`: $\frac{1}{2} \cdot \frac{1}{2} \cdot 1 \cdot 1 \cdot \frac{1}{4} \cdot 1 = \frac{1}{16}$. The unseen `the cat ate the mat .` gets exactly the same $\frac{1}{16}$: the model learned transitions, not sentences. Large language models do the same with thousands of context tokens and billions of parameters in place of a count table; the principle, self-supervised next-token prediction followed by repeated sampling, is this one.

Feature engineering versus representation learning versus self-supervision

	Feature engineering	Representation learning	Self-supervision
Who chooses the features	humans, from domain knowledge	the machine, from data	the machine, from unlabelled data via a pretext task
Needs labels	for the predictor only	yes, for the whole pipeline (unless self-supervised)	no; targets come from the data
Strength	works with little data; interpretable	finds features the designer forgot	uses observations that exist in huge quantities
Failure	designer forgets what matters	needs much data; opaque	pretext task may not transfer to the downstream task

Treating self-supervision as "supervised" or as "unsupervised" without saying why

Both labels are half right, and the professor wants the mechanism: the learning signal is a target, so training looks supervised, but the target was manufactured from the observation (the hidden word, the masked region), so no human labelled anything, which is what unsupervised means. The point to state is *where the signal comes from*. A second slip: "representation learning = deep learning". Deep learning suits it, but the idea predates it and representations can be learned by many methods.

Key insight. The representation decides what the model can see; a learner cannot use information that its input does not carry, however much data it is given.

How the professor asks it

Q. *Explain why the choice of representation can decide whether learning succeeds, using two representations of the same email, and say in which regime each one wins.*
A. A learner can exploit only the regularities visible in its input. The count-vector representation exposes "many links, no known sender, the word *prize*" as a few coordinates, so a few thousand examples suffice to find the rule; the raw character sequence hides the same facts behind thousands of characters whose positions vary from message to message, so no small dataset reveals them. With billions of emails and a model that builds its own representation, the raw input can surpass the counts because the designer of the count vector forgot features that matter. Consequence: engineered features win when data are scarce, learned representations win when data are abundant; the representation is a design choice, not a detail.

Q. *In the corpus* `the cat sat on the mat . the cat ate the rat .`, *compute the probability that the bigram model, started at* `the`, *generates* `the cat ate the mat .`, *and explain why a sentence absent from the corpus can have the same probability as one present in it.*
A. Multiply the transitions: $\mathbb{P}(\texttt{cat} \mid \texttt{the})\,\mathbb{P}(\texttt{ate} \mid \texttt{cat})\,\mathbb{P}(\texttt{the} \mid \texttt{ate})\,\mathbb{P}(\texttt{mat} \mid \texttt{the})\,\mathbb{P}(\texttt{.} \mid \texttt{mat}) = \frac{1}{2}\cdot\frac{1}{2}\cdot 1\cdot\frac{1}{4}\cdot 1 = \frac{1}{16}$. The corpus sentence `the cat sat on the mat .` also gets $\frac{1}{2}\cdot\frac{1}{2}\cdot 1\cdot 1\cdot\frac{1}{4}\cdot 1 = \frac{1}{16}$. The model learned *what follows what*, one word at a time, not whole sentences; any sequence built from seen transitions is as probable as the product of those transitions, whether or not it occurred. Consequence: generation is recombination of learned regularities, which is both its power (novel output) and its risk (fluent nonsense).

Q. *Give one self-supervised task for network traffic data and say what the learned representation could be used for afterwards.*
A. Hide one field of a flow record (say the destination port, or the byte count in one direction) and train a model to predict it from the remaining fields and the neighbouring flows of the same host. The signal is the hidden field, so no analyst labels anything. To predict it, the

model must learn what normal combinations of protocol, timing and volume look like, which is a representation of "typical traffic". Afterwards a small labelled set of attack and benign flows adapts the representation to classification, and a flow that the model reconstructs badly is a candidate anomaly. Consequence: labels, the scarce resource in security, are needed only for the last step.

6 Four ingredients of a learning system

Experience, model, learning algorithm, evaluation criterion

Almost any learning system can be examined through four questions.

Experience	What information is available?
Model	In what form can acquired knowledge be represented, and what does it output?
Learning algorithm	How does experience change the model?
Evaluation criterion	How is success judged?

This begins to answer "what gives a machine the capacity to learn?": the model determines what can be represented; the learning algorithm is the mechanism through which experience changes it; the evaluation criterion indicates which changes are useful; the experience determines what evidence exists in the first place. Representation learning adds that the internal representation can itself be shaped by experience.

Purpose. To describe, criticise and improve any application without naming an algorithm.

Impact. Each ingredient is responsible for a different failure: bad experience, a model that cannot express the relationship, an algorithm that cannot find it, or a criterion that rewards the wrong thing. Diagnosing a failing system means asking which ingredient is at fault.

Four systems, four ingredients

	Experience	Model	Learning algorithm	Evaluation
Bigram text model	a corpus	a table of next-word probabilities	count and normalise	probability assigned to held-out text
Digit classifier	labelled images	a function from pixels to 10 probabilities	adjust parameters to reduce error on examples	accuracy on new images
Fraud detector	past transactions with confirmed outcomes	a function from transaction features to a fraud probability	the same, with rare positives weighted up	cost of missed fraud plus cost of blocked customers
Warehouse robot	its own actions in simulation	a policy from sensor readings to motor commands	reinforcement from rewards	task completion and safety in the real warehouse

Each row can be discussed without naming an algorithm; that is the point of the checklist.

Designing an application from scratch: the irrigation pump

Setup. Predict, from a farm's sensor data, whether an irrigation pump will fail in the next week.

Held fixed. The question (failure within seven days) and the deployment (the maintenance team ranks pumps weekly).

What we manipulate. The four ingredients, written in the recommended order: (1) *evaluation criterion*: on pumps and months not used during development, how many failures were flagged in advance and how many unnecessary inspections were triggered, converted into money; (2) *experience*: two years of hourly readings (current, vibration, temperature, flow) from 300 pumps, with the maintenance log giving each failure and its date; each week of readings becomes an input and the label is whether a failure occurred in the following week; (3) *model output*: a probability of failure within seven days, so pumps can be ranked; (4) *learning algorithm*: its signal is the disagreement between predicted probability and what happened, aggregated over many pump-weeks, with missed failures weighted far above false alarms because failures are rare.

What we measure. The criterion in (1), on held-out pumps and later months.

What the plot shows. Not a curve but a checklist; the professor's question is "map each part to an ingredient".

How to read it. Start with the criterion (what would make the client say "it works"?), then the experience (what is logged, what can be labelled, what changes over time), then the model output (number, class, probability, ranking, sentence?), and only then the algorithm, which has the most textbooks and the least influence on whether the project makes sense.

Starting from the algorithm

The tempting answer to "design a system for X" is "use a random forest" or "use a neural network". It earns nothing because it says nothing about what the system will see, what it outputs, or how anyone will know it works. The fix is the order criterion, experience, output, algorithm. A second slip is to confuse the model's *output* (a probability of failure) with the *decision* (inspect the top ten pumps): the output is the model ingredient, the inspection policy is a criterion applied to it.

Key insight. Any learning system is experience plus model plus algorithm plus criterion; the capacity to learn lives in the last three and the evidence lives in the first.

How the professor asks it

Q. *Describe an application not used in the chapter without naming any algorithm, and map each part of your description to one of the four ingredients. (Exercises ch. 1, Q9)*
A. Hourly electricity demand of a city for the next day. (a) Experience: years of hourly demand with weather, calendar and events. (b) Model output: a number for each of the 24 hours, possibly an upper quantile since under-supply costs more than over-supply. (c) Signal for improvement: the gap between predicted and realised demand on past days, with the asymmetric cost built in. (d) Success: error on *later* days not used for training, weighted by that cost, against the baseline "same as last week". Mapping: (a) experience, (b) model, (c) learning algorithm, (d) evaluation criterion. Design in the order (d), (a), (b), (c).

Q. *Explain why the evaluation criterion should be written before the learning algorithm is chosen, with an example where choosing the algorithm first leads to a useless system.*
A. The criterion states what the client means by "it works"; the algorithm is a means of producing a model that scores well on that criterion, so without the criterion there is nothing to optimise and no way to compare candidates. Example: a fraud team picks a high-accuracy classifier first; with 0.5% fraud, the rule "never fraud" already scores 99.5% accuracy and the chosen model is tuned to match it, blocking nobody. Had the criterion been written first (cost of missed fraud plus cost of blocked customers, on later transactions), the team would have weighted rare positives up and judged the model on the money it saves. Consequence: the algorithm has the most textbooks and the least influence on whether the project makes sense.

Q. *A learned system performs badly. Name, for each of the four ingredients, one way that*

ingredient alone could be the cause.
A. Experience: the examples are narrow, mislabelled or outdated, so the regularities learned do not hold at deployment (phishing from one feed). Model: the family cannot express the relationship (a straight-line model for a curved effect), so no amount of data helps. Learning algorithm: it fails to find a good solution of its own objective (stops too early, gets stuck), so the model is worse than the family allows. Evaluation criterion: it rewards the wrong thing (accuracy on 0.5% fraud), so a useless model looks excellent. Consequence: diagnosis means isolating the ingredient, because the fixes (collect, enlarge the family, optimise better, change the metric) are entirely different.

7 The challenge of generalization

Generalization

Experience is always finite (no digit recogniser sees every handwriting, no medical system every patient, no robot every situation) and the deployed system meets *new* inputs. **Generalization** is the ability of a learned system to perform well in new situations that were not encountered during learning. The central challenge of machine learning is

$$\text{Finite Experience} \xrightarrow{\text{Learning}} \text{Ability Useful in New Situations.}$$ [not on your sheet: know it]

Purpose. It is the actual goal; performance on the training examples is a side effect, and a dangerous one to optimise for.

Impact. Chapter 2 will give it two numbers, the training risk $\widehat{R}$ (computable) and the population risk R (wanted); Chapter 3 studies their gap. Here the point is qualitative: training performance is not evidence of generalization.

Finite data are consistent with infinitely many rules

Proposition. Let $\mathcal{X}$ be an infinite input space, $\mathcal{Y}$ have at least two elements, and $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ be any finite dataset. Then infinitely many functions $f : \mathcal{X} \to \mathcal{Y}$ agree with the data, and two of them may disagree on *every* input not in the data. [not on your sheet: know it]
Proof, step by step. Let $S = \{x^{(1)}, \dots, x^{(N)}\}$, a finite set, so $\mathcal{X} \setminus S$ is infinite (an infinite set minus a finite one stays infinite). Fix two distinct outputs $y_0 \neq y_1$ in $\mathcal{Y}$. For each subset $T \subseteq \mathcal{X} \setminus S$ define

$$f_T(x) = \begin{cases} y^{(i)} & \text{if } x = x^{(i)} \in S, \\ y_1 & \text{if } x \in T, \\ y_0 & \text{otherwise.} \end{cases}$$

Every f_T agrees with the data, because on S it returns the recorded label. Distinct subsets $T \neq T'$ give distinct functions, because they differ on any x belonging to one of them and not the other. An infinite set has infinitely many subsets, so there are infinitely many data-consistent functions. Finally $T = \mathcal{X} \setminus S$ and $T' = \emptyset$ give two data-consistent functions that disagree on every input outside the data. □
What it means. Off the training inputs, the data say nothing at all. Whatever a learned system does there comes from its *model* and its *learning algorithm*, that is, from assumptions built in by the designer. Generalization is never a property of the data alone; it is a property of the match between those assumptions and the world.

The perfect memoriser

"Look up x in the training set and return its label; otherwise return class 0." Zero error on every training example; nothing learned. On a continuous input space a new x is never in the table, so the rule always answers 0, and with two balanced classes it is right half the time: a coin toss. A learner evaluated only on its training data cannot tell this lookup table from a genuinely useful model. This is why performance on previously encountered examples gives only part of the evidence about what has been learned.

The shortcut experiment: wolves and snow

Setup. A system distinguishes photographs of wolves from dogs. In the collection, most wolf photographs contain snow and most dog photographs do not. Say 1000 wolves of which 900 in snow, 1000 dogs of which 100 in snow.

Held fixed. The learner, the labels, the task.

What we manipulate. The *conditions* of the test images: first the developers' held-out images (same collection, same correlation), then deployment images where users photograph dogs in snow and wolves in summer (say 30% snow in both classes), then two probe images, a husky in snow and a wolf on green grass.

What we measure. Accuracy under each condition, for the learner and for the explicit rule "white background ⇒ wolf".

What the plot shows. Accuracy against condition. The snow rule scores $(900 + 900)/2000 = 90\%$ on the collection and the learner scores similarly on the developers' held-out split (it shares the correlation). At deployment with 30% snow in both classes the snow rule scores $0.5 \cdot 0.3 + 0.5 \cdot 0.7 = 50\%$, a coin toss, and the learner's accuracy collapses with it. On the two probes a system that learned wolves gets both right; a system that learned snow gets both wrong.

How to read it. (1) *Initial success*: the proposition above guarantees that alternative rules agree with the data; "snow ⇒ wolf" is one, and far easier to detect than wolf anatomy, so the learner adopted it. (2) *Failure*: the correlation was a property of the *collection*, not of wolves, so the rule is systematically wrong once the collection's conditions change. (3) *Diagnosis*: decouple the cue from the label and look. Nothing here opens the model; the reasoning uses only the definition of generalization.

Quantity, diversity, representativeness

Three different properties of a collection of experience. **Quantity**: how many examples. **Diversity**: how much of the space of situations they cover. **Representativeness**: whether they are distributed like the situations the deployed system will meet. A learner finds regularities in whatever it is given; a million near-repeats of one situation contain one situation's regularities, which will not hold elsewhere.

Purpose. To separate "more data" from "better experience".

Impact. More data lowers the error of estimating regularities *present* in the collection and does nothing about regularities *absent* from it; generalization depends on the second.

Three stories about experience

Birds. Dataset A: one million photographs from one camera in one park. Dataset B: fifty thousand by many people in many countries, seasons, weathers, backgrounds and angles. For a system used by the public, B: A's images share lighting, background, altitude and season, so the regularities a learner finds in A (this park's foliage, this camera's colour balance) will not hold elsewhere, exactly as with the snow. A would be preferable if the system is deployed *in that park with that camera*, where A is perfectly representative, or if the species are so similar that

sheer numbers are needed and the deployment conditions match. *The robot in the wrong world.* A warehouse robot trained only in a simulator with flat floors, identical friction and constant lighting fails in reality. If only the simulator can be changed, vary what varies in reality *and affects the decisions*: floor slope and bumps, box weights and friction, lighting, sensor noise, worker behaviour, wheel slippage. A policy trained across the range cannot rely on any one constant. Making the simulator visually realistic would not help by itself: if the floor is still flat, the robot still learns that wheels never slip. *The advertisement.* "Our AI has learned every possible situation from millions of examples." This confuses situations that were *observed* with situations that *may occur*. Millions of examples are a finite sample; the situations a deployed system faces are not a fixed list, include ones nobody recorded, and the world changes after collection. The honest claim is good performance on seen situations and an *expectation* that the regularities carry over; whether they do is generalization, and no amount of data makes the question disappear.

Shortcut versus target leakage

Target leakage: a feature that would *not be available at decision time*, or that is a consequence of the label (an incident ticket number, "host quarantined afterwards", "number of analyst comments"). It predicts the label almost perfectly in the dataset and does not exist for a new input. Fix: remove the field. **Shortcut**: a feature that *is* available at decision time but whose association with the label is an artefact of how the data were collected (snow, hour of day, hospital tag, camera). The feature means something else at deployment. Fix: collect data where the association is broken and test on them; dropping the feature is one option, not the only one.

Purpose. The two look identical in a table of accuracies (both inflate it) and need different repairs.

Impact. Leakage disappears when the field is removed; a shortcut survives feature removal if another collection artefact remains, and is only exposed by data gathered under different circumstances.

Testing for generalization without opening the model

Setup. Any fitted model, suspected of relying on collection-specific cues.

Held fixed. The model (no retraining in steps 1 to 3).

What we manipulate. The test conditions: (1) *hold out*: evaluate on examples the learner never saw (Chapter 2 makes this validation and test sets); (2) *break the shortcuts*: construct test inputs where suspected spurious cues are decoupled from the label (dog in snow, wolf on grass, attack by day); (3) *shift the conditions*: evaluate on data from a different camera, hospital, country or year; (4) *ablate*: retrain without the suspected feature and compare.

What we measure. Error under each condition; the *drop* between the original held-out error and the shifted or shortcut-breaking error.

What the plot shows. A bar per condition. A model that learned the task shows small drops; a model that learned the collection shows a large drop under (2) and (3). Step (4) alone is suggestive, not conclusive: if the validation split comes from the same collection, the shortcut holds on both sides of the split and the ablation measures how much the model leans on the feature, not whether the feature is stable.

How to read it. A large drop reveals reliance on regularities that held in the experience but not in the world. Also list the situations absent from the experience: the model's behaviour there is an assumption, not a learned fact.

Memoriser versus shortcut learner versus genuine learner

	Memoriser	Shortcut learner	Genuine learner
Training error	zero	near zero	small
Held-out, same collection	bad (coin toss off the table)	good (the correlation is still there)	good
Shifted or shortcut-breaking test	bad	bad, systematically wrong	good
What it learned	the table	a collection artefact	a regularity of the world
Caught by	any held-out set	only decoupled or shifted data	nothing to catch

"One million is more than fifty thousand" and "zero training error is an achievement"

Quantity is not coverage: a billion photographs of one park are one situation. Zero training error is compatible with having learned nothing (the memoriser) or the wrong thing (the snow detector); it is a reason to look at new data, not a result. And calling a shortcut "leakage" loses the mark: leakage is unavailable at decision time and is fixed by removal; a shortcut is available, spuriously associated, and fixed by breaking the association in the data and testing on it.

Key insight. Off the training inputs the data say nothing; what a learner does there comes from its built-in assumptions, and a shortcut is what happens when the assumption "cues that worked in the collection work in the world" is false.

How the professor asks it

Q. *A wolf-versus-dog classifier performs extremely well for its developers and badly after deployment; most training wolves stood in snow. Explain the initial success, the failure, and a diagnosis, without assuming anything about the model's internals. (Exercises ch. 1, Q4)*
A. Initial success: "white background ⇒ wolf" agrees with almost all training examples (finite data admit many consistent rules) and is far simpler to detect than wolf anatomy, so the learner adopted it; the developers' held-out images came from the same collection and shared the correlation, so it scored well there too. Failure: users photograph dogs in snow and wolves in summer; the correlation was a property of the collection, not of wolves, so the rule is now systematically wrong. Diagnosis: two photographs, a husky in snow and a wolf on green grass; a system that learned wolves gets both right, one that learned snow gets both wrong. Only the definition of generalization was used: good on seen situations, poor on new ones, because a regularity held in the experience and not in the world.

Q. *Most attacks in the training data occurred at night and most benign flows by day. Explain how "hour of day" becomes a shortcut, why this is not target leakage, and what you would do about it. (Tutorial 1, Q1(e))*
A. In the collected data the hour predicts the label well, so the model can use it instead of learning what attack traffic looks like; at deployment a daytime attack is missed and night-time backups raise alerts. It is a shortcut, not leakage: the hour *is* available when a new flow is scored and is not caused by the label; the association was created by the collection (a red-team campaign run overnight) and is not stable. Leakage would be a field like the incident ticket number, absent for a new flow and fixed by removal. For the shortcut, collect or construct data in which the hour varies independently of the label (attacks by day, benign traffic by night), test on them, and consider removing the hour only if nothing stable is left in it.

Q. *Removing "hour of day" raises validation error from* 0.04 *to* 0.21. *On a later week with attacks in office hours, the full model scores* 0.35 *and the model without the hour* 0.08. *Interpret both results and explain why the ablation alone could not have raised the alarm.*

A. The ablation shows the model leans on the hour far more than on any flow property, which is suspicious (the hour describes the capture, not the flow) but not proof: a stable feature would also hurt when removed. The later week settles it: 0.35 against 0.08 is a shortcut costing 0.27 at deployment while it gained 0.17 on the original split. The ablation could not see this because the validation set came from the same collection, where "night ⇒ attack" held on both sides of the split; it measured reliance, not stability. Only data collected under different circumstances separate a stable regularity from an artefact.

8 A connection to information theory

Entropy, cross-entropy, Kullback–Leibler divergence

For distributions p and q on the same C outcomes, with $q_c > 0$ wherever $p_c > 0$ and the convention $0 \log 0 = 0$:

$$H(p) = -\sum_c p_c \log p_c \text{ [on your sheet]}, \qquad H(p,q) = -\sum_c p_c \log q_c \text{ [on your sheet]},$$

$$\mathrm{KL}(p \,\|\, q) = \sum_c p_c \log \frac{p_c}{q_c} \text{ [on your sheet]}.$$

$H(p)$, the **entropy**: how uncertain the truth is; $-\log p_c$ is the surprise of outcome c, and $H(p)$ is the average surprise when you believe the truth and the truth happens. $H(p,q)$, the **cross-entropy**: the average surprise when you believed q and the truth was drawn from p. $\mathrm{KL}(p\|q)$, the **divergence**: the extra surprise caused by believing q instead of p. Expanding the divergence gives the identity

$$H(p,q) = H(p) + \mathrm{KL}(p \,\|\, q) \text{ [on your sheet]}.$$

Base-2 logarithms give bits, natural logarithms (customary in machine learning) give nats; the choice multiplies everything by a constant.

Purpose. A numerical way to say how good a set of predicted probabilities is, and a vocabulary for how much information a representation keeps.

Impact. Cross-entropy will be the training criterion for classification (Chapter 2); the identity says its floor is $H(p)$, set by the problem, and the part a model can reduce is the divergence.

Gibbs' inequality: $\mathrm{KL}(p\|q) \geq 0$ with equality iff $p = q$

Step 1, a one-line inequality about logarithms. For every $t > 0$, $\ln t \leq t - 1$ with equality iff $t = 1$ [on your sheet]. Proof: $g(t) = t - 1 - \ln t$ has $g'(t) = 1 - 1/t$, negative on $(0,1)$ and positive on $(1,\infty)$, so g has its unique minimum at $t = 1$, where $g(1) = 0$; hence $g \geq 0$. Check: $t = 0.5$ gives $\ln 0.5 = -0.693 \leq -0.5$; $t = 2$ gives $0.693 \leq 1$.
Step 2, apply it term by term and average under p. Let $S = \{c : p_c > 0\}$. Then

$$-\mathrm{KL}(p\|q) = \sum_{c\in S} p_c \ln \frac{q_c}{p_c} \;\leq\; \sum_{c\in S} p_c\Big(\frac{q_c}{p_c} - 1\Big) = \sum_{c\in S} q_c - \sum_{c\in S} p_c = \sum_{c\in S} q_c - 1 \;\leq\; 0,$$

where the first inequality is Step 1 with $t = q_c/p_c$, the next equality cancels p_c against $1/p_c$, the sum of p_c over S is 1, and $\sum_{c\in S} q_c \leq 1$ because q is a distribution. Hence $\mathrm{KL}(p\|q) \geq 0$. Changing the base multiplies by a positive constant, so the sign is unaffected.
Step 3, equality. Equality needs equality in Step 1 for every $c \in S$, i.e. $q_c = p_c$ wherever $p_c > 0$, and $\sum_{c\in S} q_c = 1$, which forces $q_c = 0$ wherever $p_c = 0$. So $q = p$. Conversely $p = q$ gives every term $\log 1 = 0$. □
Consequences. $H(p,q) = H(p) + \mathrm{KL}(p\|q) \geq H(p)$ with equality iff $q = p$: among all predictions q, the expected surprise is smallest when q is the truth. With q uniform, $\log C - H(p) =$

$\mathrm{KL}(p\|\mathrm{unif}) \geq 0$, so the uniform distribution has the largest entropy on C outcomes, $\log C$. The whole theorem is the tangent-line inequality $\ln t \leq t - 1$ applied to the ratios q_c/p_c and averaged under p; "bound the log by its tangent, then average" recurs throughout machine learning.

The calibration experiment: what cross-entropy rewards

Setup. A binary outcome whose true distribution is $p = (0.7, 0.3)$ (say 70% of flows at this x are benign). A forecaster announces q.

Held fixed. The truth p.

What we manipulate. The announced q: the truth, hedging, overconfidence in the right direction, the wrong direction, extreme overconfidence.

What we measure. The average surprise $H(p, q)$ over many draws from p, and the divergence $\mathrm{KL}(p\|q) = H(p, q) - H(p)$, in bits.

What the plot shows. $H(p, q)$ against q_1: a U-shaped curve with its minimum $H(p) = 0.881$ bits at $q_1 = 0.7$, rising to $+\infty$ at both ends.

Prediction q	$H(p, q)$	$\mathrm{KL}(p\|q)$	Reading
$(0.7, 0.3)$	0.881	0	the truth: minimum possible
$(0.5, 0.5)$	1.000	0.119	hedging costs 0.119 bits
$(0.9, 0.1)$	1.103	0.222	overconfident in the right direction: worse than hedging
$(0.3, 0.7)$	1.370	0.489	wrong direction
$(0.99, 0.01)$	2.003	1.122	the 0.3 mass pays $\log_2 100 = 6.64$ bits each time

How to read it. The floor 0.881 is a property of the *problem*: no prediction beats it because the outcome is genuinely uncertain. Overconfidence is punished sharply: $(0.9, 0.1)$ favours the right class and still scores worse than the uninformative $(0.5, 0.5)$. Minimising cross-entropy therefore trains *calibrated* probabilities, not merely the right argmax. In nats the same table reads 0.611, 0.693, 0.765, 0.950, 1.389.

Why this matters for learning

Cross-entropy as a criterion. A classifier assigns probabilities to several classes; cross-entropy compares them with what actually occurs: high probability on the correct class costs little, low probability costs much. For a *one-hot* truth ($p_y = 1$) it reduces to $-\log q_y$ [on your sheet], the penalty for the probability given to the observed class: probabilities 0.9, 0.5, 0.1 on the true class cost 0.105, 0.693, 2.303 nats. By Gibbs, the expected penalty is minimised exactly when the predicted probabilities equal the true conditional probabilities, which is why a familiar information quantity became the workhorse loss of classification. *Representations and information.* Raw observations contain many kinds of information, only some useful for the purpose; an informative representation retains what is relevant about the output and discards the rest.

Entropy versus cross-entropy versus divergence

	$H(p)$	$H(p, q)$	$\mathrm{KL}(p\|q)$
Depends on	the truth only	truth and prediction	truth and prediction
Measures	uncertainty of the outcome	average surprise of a believer in q	extra surprise caused by believing q
Minimum	0 for a certain outcome; $\max \log C$ for uniform	$H(p)$, at $q = p$	0, at $q = p$
Symmetric	not applicable	no	no: $\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$ in general
Role in learning	the irreducible floor	the training loss	the part the model can reduce

Three slips with entropies

(1) Quoting KL with the arguments swapped: it is not symmetric; check the order. (2) Predicting probability exactly 0 for an outcome that can happen: $q_c = 0$ with $p_c > 0$ gives $H(p, q) = +\infty$. (3) Thinking that the argmax is all that matters: cross-entropy penalises overconfidence, so $(0.9, 0.1)$ loses to $(0.5, 0.5)$ when the truth is $(0.7, 0.3)$; a model trained with it is pushed towards calibrated probabilities, which Chapter 2 and Chapter 4 rely on.

Key insight. $H(p, q) = H(p) + \mathrm{KL}(p\|q)$ with $\mathrm{KL} \geq 0$: the expected surprise of a prediction is the problem's own uncertainty plus a penalty that vanishes only when the prediction is the truth.

How the professor asks it

Q. *State Gibbs' inequality, prove it, and explain what it implies about minimising cross-entropy.*
A. $\mathrm{KL}(p\|q) \geq 0$ with equality iff $q = p$. Proof: $\ln t \leq t - 1$ for $t > 0$ (the function $t - 1 - \ln t$ has derivative $1 - 1/t$, so its minimum is 0 at $t = 1$); apply it to $t = q_c/p_c$ and average under p: $-\mathrm{KL} = \sum_c p_c \ln(q_c/p_c) \leq \sum_c (q_c - p_c) = \sum_c q_c - 1 \leq 0$; equality forces $q_c = p_c$ wherever $p_c > 0$ and the remaining q_c to be 0. Since $H(p, q) = H(p) + \mathrm{KL}(p\|q)$, cross-entropy is minimised exactly at $q = p$ with minimum value $H(p)$: minimising expected cross-entropy pushes a classifier towards the true conditional probabilities, which is why it is the training criterion for classification.

Q. *The truth is* $(0.5, 0.5)$ *and a model predicts* $(0.99, 0.01)$. *Compute the cross-entropy in bits, compare with the honest prediction, and say what happens for the prediction* $(1, 0)$.
A. $H(p, q) = -0.5 \log_2 0.99 - 0.5 \log_2 0.01 = 0.5(0.0145) + 0.5(6.644) \approx 3.33$ bits, against $H(p) = 1$ bit for $q = p$: overconfidence more than triples the penalty, because the half of the draws that land on the second outcome each pay $\log_2 100$ bits. For $(1, 0)$, $\log_2 0 = -\infty$ and $H(p, q) = +\infty$: never predict exactly 0 for something that can happen. Consequence: a loss that explodes on confident mistakes trains models to hedge in proportion to their real uncertainty.

Q. *Explain why* $H(p)$ *is a floor that no model can beat, and what that means for a reported error on a problem with genuinely uncertain outcomes.*
A. By Gibbs, $H(p, q) = H(p) + \mathrm{KL}(p\|q) \geq H(p)$ for every q, so the smallest achievable expected surprise is the entropy of the truth, reached only by predicting the truth itself. The floor is a property of the problem (how random the outcome is given the input), not of the learner. Consequence: on a problem where 30% of flows at a given x are attacks, a cross-entropy of 0.611 nats is perfect, not mediocre; a model reporting far below the floor on its training data has memorised noise, and a claim of near-zero loss on an inherently uncertain outcome should be disbelieved.

Coverage of the textbook chapter

Textbook item	Where here
Learning, informal; four abilities; radiologist example	Section 1
Route 1, Pascaline, expert systems, MYCIN, bottleneck; programs that should stay programs; knowledge engineer's year; the "3" thought experiment; ability $\neq$ procedure; route 2; deciding between routes; gotcha	Section 2
Three forms of experience; three settings; hospital four projects; same records three questions; "unsupervised" gotcha	Section 3
Supervised example and dataset; where labels come from; phishing; training cycle figure; regression and classification; ten tasks; structured outputs; prediction, decision, action; screening system; gotchas	Section 4
Feature engineering; two representations of an email; representation definition and pipelines; self-supervision; three tasks; bigram example and generation; two-branch figure	Section 5
Four ingredients; four systems table; irrigation pump; writing the ingredients in order	Section 6
Generalization; proposition and proof; memoriser; wolves and snow; quantity versus diversity; robot; advertisement; testing without opening the model; gotchas	Section 7
Entropy, cross-entropy, KL, identity; Gibbs with proof; numbers table; why it matters; computing with entropies; classic questions	Section 8
Notes on the lecture notes (stray citation marker, "ground truth", four abilities not a taxonomy, generalization deferred, information theory recalled without proof)	the warnings of Sections 1, 4 and 8

The ten mistakes of this chapter

1. Answering "learn it" to every task: tax, sorting and password checks have a known procedure and need exactness.
2. "Machine learning replaces programming" or "expert knowledge is obsolete": only the task-specific rules are replaced; experts choose what to measure and cover cases with no data.
3. Naming the paradigm from the data type: four X-ray projects, four paradigms; the feedback decides.
4. "Unsupervised means no objective": there is always a criterion, only no human answer per example.
5. Trusting "ground truth": labels are measured, annotated or observed later, and their error caps the achievable accuracy.
6. "Regression = linear", "classification = binary": the words name output types.
7. Reading a probability as a decision: the decision needs costs; the same 0.6 decides differently under different costs.
8. "A million examples beat fifty thousand": quantity is not diversity or representativeness;

a million photos of one park are one situation.

9. Zero training error as an achievement, and confusing a shortcut (available, spurious; fix by breaking the association) with leakage (unavailable at decision time; fix by removal).
10. Entropy slips: swapped KL arguments, probability exactly 0, and forgetting that cross-entropy punishes overconfidence, so it trains calibrated probabilities rather than just the right argmax.

Chapter 1 in ten lines

1. Learning = capacity to learn + experience → ability; abilities: recognise, predict, reason, act (vocabulary, not taxonomy).
2. Route 1 writes the rules (procedure known, exactness required); route 2 learns from experience (correct behaviour recognisable, procedure not describable). Ability ≠ procedure.
3. Expert systems hit the knowledge acquisition bottleneck: obtaining, representing, unfamiliar situations; learning attacks all three, experts still matter.
4. Experience is examples, observations or interactions; the feedback that reaches the learner names the paradigm: supervised, unsupervised or self-supervised, reinforcement.
5. Supervised pairs (x, y) with labels from measurement, annotation or delayed outcomes, each imperfect; the output type decides regression versus classification; prediction, decision and action are three objects.
6. The representation decides what the model sees: engineered features win with little data, learned representations with much; self-supervision manufactures targets from the data; next-token prediction sampled forward is generation.
7. Four ingredients: experience, model, learning algorithm, evaluation criterion; design in the order criterion, experience, output, algorithm.
8. Generalization is the goal; finite data agree with infinitely many rules, so off-sample behaviour comes from the designer's assumptions.
9. Shortcuts: a collection artefact (snow, hour of day) fits the sample and fails in the world; catch it with decoupled or shifted test data; shortcut ≠ leakage.
10. $H(p, q) = H(p) + \mathrm{KL}(p\|q)$ with $\mathrm{KL} \geq 0$ (Gibbs: $\ln t \leq t - 1$, then average): cross-entropy is minimised by the truth, has floor $H(p)$, punishes overconfidence, and trains calibrated probabilities.